

DATA STRUCTURES AND ALGORITHMS

Practical Project Report

Project Title

Library Management System

A Linked List, Queue, Stack and Merge Sort based solution

Team Member	ID Number
Fanuel Aregay	CEP0053/17
Abrham Nega	CEP0008/17
Aman Haftu	CEP0015/17
Habtamu Gizaw	CEP0064/17
Fikremariam Belete	CEP0056/17

Submission Date: August 28, 2026

Table of Contents

1. Project Objective	3
2. Project Requirements	4
3. Problem Statement.....	5
4. Data Structures and Algorithms	6
5. Implementation	8
6. Analysis and Evaluation	13
7. Appendix.....	16

1. Project Objective

The aim of this DSA project is to design and implement a set of core data structures and algorithms to solve a real-world problem faced by libraries, schools, and small organizations: keeping track of a growing book collection, managing who has borrowed which book, and recording the history of returns in an organized and efficient way. Manual, paper-based, or simple spreadsheet-based tracking of books quickly becomes unmanageable as a collection grows, leading to lost records, duplicate entries, and no reliable way to know which books are available at any given moment.

This project, the Library Management System (LMS), was built to directly apply the concepts taught in the Data Structures and Algorithms course to a problem that a library administrator would recognize immediately. Rather than using a database engine or an existing framework, the system implements its own dynamic data structures from first principles in C++, which forces a deeper understanding of memory management, pointer manipulation, and the trade-offs between different structures.

The specific objectives of the project are:

- To design a dynamic catalog of books that can grow or shrink at run time without a fixed size limit, using a singly linked list.
- To provide fast, well-organized operations for adding, searching, updating, deleting, and listing books.
- To model the natural real-world order of borrowing requests using a First-In-First-Out (FIFO) queue, so that requests are processed fairly in the order they were made.
- To model the return history using a Last-In-First-Out (LIFO) stack, so that the most recently returned book can always be inspected first.
- To implement a divide-and-conquer sorting algorithm (merge sort) that operates directly on the linked list, allowing the catalog to be sorted by publication year without converting it into an array.
- To persist the catalog to disk between program runs, so that the library's data is not lost when the application closes.
- To reinforce, through hands-on implementation, the theoretical concepts of time and space complexity, recursion, and pointer-based data structures covered in the DSA course.

By the end of the project, the team is able to demonstrate a working console application that a small library could realistically use to manage its day-to-day circulation activities, while also being able to explain and justify every design decision in terms of the underlying data structures and algorithms.

2. Project Requirements

The Library Management System is required to solve the stated real-world problem using at least two distinct data structures and algorithms working together. Concretely, the system must satisfy the following functional and non-functional requirements:

2.1 Functional Requirements

- Add a new book to the catalog with a unique ID, title, author, and publication year.
- Prevent duplicate book IDs from being added to the catalog.
- Display the full list of books currently in the catalog, including their availability status.
- Search for a book by its ID and display its details.
- Update the title, author, and year of an existing book.
- Delete a book from the catalog by its ID.
- Allow a member to place a borrow request for an available book, recording it in a borrow queue in the order it was made.
- Allow a borrowed book to be returned, updating its availability and recording the return at the top of a history stack.
- Sort the entire catalog by publication year using a divide-and-conquer algorithm.
- Persist the catalog to a text file so that data survives between sessions.

2.2 Non-Functional Requirements

- The system must run as a lightweight, dependency-free console application written in standard C++.
- Core catalog operations must use a linked list rather than a fixed-size array, so the catalog is not limited by a pre-declared capacity.
- The sorting algorithm must be a divide-and-conquer algorithm operating directly on the linked list, without requiring the whole catalog to be copied into an array first.
- The borrow queue and return history may use a fixed-capacity array internally, since the number of simultaneously outstanding requests is naturally bounded in a small library.
- The interface must be simple enough for a non-technical library assistant to operate through a numbered menu.
- The design should be modular, with each data structure encapsulated in its own class, so that components can be tested, replaced, or extended independently.

3. Problem Statement

Small libraries, departmental reading rooms, and personal book collections often lack any dedicated software to track their inventory. Books are added by hand to notebooks or spreadsheets, borrow and return activity is remembered informally, and there is no simple way to answer basic questions such as “which books are currently available?” or “who is next in line to borrow this book?”. This leads to books being lost, borrow requests being handled unfairly (out of order), and no historical record of returns for auditing purposes.

The Library Management System addresses this problem by providing a single console application that models the catalog, the borrowing process, and the return history using appropriate data structures, and that keeps this information consistent and persistent.

3.1 Inputs

- Book records entered by the librarian: numeric ID, title, author, and publication year.
- Menu selections and book IDs entered by the user during a session (add, search, update, delete, borrow, return, sort, display).
- An existing books.txt file, if present, containing previously saved catalog data as one field per line.

3.2 Outputs

- An up-to-date, formatted listing of all books and their availability status.
- Confirmation or error messages for every operation (e.g. duplicate ID rejected, book not found, book updated).
- The contents of the borrow queue and the return history stack, on demand.
- A books.txt file that reflects the current state of the catalog after every change, so it can be reloaded on the next run.

3.3 Constraints and Assumptions

- Each book ID is assumed to be unique; the system rejects an add operation if the ID already exists.
- The system is single-user and single-session; it does not need to handle concurrent access from multiple terminals.
- Persistence is implemented with a simple flat text file (five lines per book) rather than a full database engine, which is an accepted simplification for the scope of this course project.
- The linked list is singly linked, so traversal is only possible from head to tail, which is factored into the design of the merge sort and the tail-insertion logic.
- The borrow queue and return history are capped at 100 entries by their fixed array size; exceeding this bound is reported to the user rather than silently failing.

4. Data Structures and Algorithms

Three data structures and one divide-and-conquer algorithm were chosen to model the three natural parts of the library problem: the catalog itself, the queue of borrow requests, and the history of returns. The catalog is built on a singly linked list because it must grow without limit; the borrow queue and return history are built on fixed-capacity arrays because the number of requests a small library needs to track at once is naturally bounded.

4.1 Singly Linked List – Book Catalog (mandatory)

The catalog of books is implemented as a singly linked list of Node objects, each holding a Book record (id, title, author, year, availability) and a pointer to the next node. A linked list was chosen over a fixed-size array because the number of books in a real library is not known in advance and changes constantly as books are added or withdrawn. Using a linked list means memory is allocated one node at a time as books are added, with no risk of the catalog overflowing a pre-declared capacity, and no need to shift elements when a book in the middle of the list is deleted, unlike with an array-based list.

4.2 Merge Sort – Divide and Conquer (mandatory)

To sort the catalog by publication year, the project implements merge sort directly on the linked list rather than copying the list into an array, sorting the array, and copying it back. Merge sort was chosen specifically because it is well suited to linked lists: unlike quicksort, it does not need random access to elements, and unlike simple quadratic sorts (bubble sort, insertion sort, selection sort), it guarantees $O(n \log n)$ performance even in the worst case.

The implementation follows the classic divide-and-conquer pattern:

- Divide: the `split()` function uses the slow/fast pointer technique — fast advances two nodes for every one step of slow — to find the middle of the list in a single pass, dividing it into a left half and a right half.
- Conquer: `mergeSort()` is called recursively on each half until a sub-list of length 0 or 1 is reached, which is trivially sorted (the recursion's base case).
- Combine: the `merge()` function compares the front of each sorted half and recursively re-links the smaller node's next pointer to the merge of the remainder, splicing the two halves together in ascending year order without allocating any new nodes.

Because `merge()` re-links existing nodes instead of copying data, the algorithm sorts the catalog in $O(n \log n)$ time using only $O(\log n)$ additional space for the recursive call stack, which is a meaningful improvement over the $O(n^2)$ time of the simpler sorts typically covered earlier in the course. This also means the Book records themselves are never moved in memory during a sort — only next pointers change.

4.3 Array-Based Queue – Borrow Requests

Borrow requests are modelled with a FIFO queue (BorrowQueue), implemented as a fixed-size array of up to 100 BorrowRequest records (book ID and borrower name) together with front and rear index counters. When a member borrows an available book, a request is appended at the rear index and rear is advanced; a corresponding `dequeue()` method is provided to remove a request from the front. A queue is the natural

structure here because fairness matters: the first person to request a book should logically be the first one served, which is exactly the FIFO ordering a queue guarantees.

An array was chosen over a linked list for this structure because a small library only needs to track a bounded number of outstanding requests at any one time (capped at 100 in this implementation), so the simplicity and cache-friendly layout of a plain array outweighs the benefit of unlimited growth that a linked-list queue would offer. Both `enqueue()` and `dequeue()` run in $O(1)$ time because they only touch the rear or front index, never scanning the array.

4.4 Array-Based Stack – Return History

The history of returned books is modelled with a LIFO stack (`ReturnStack`), implemented as a fixed-size array of up to 100 `Book` records together with a single `top` index. Every time a book is returned, its record is pushed onto the array at position `top + 1`. A stack is the appropriate structure for a history log because the most common query on a history is “what was returned most recently?”, which a stack answers in $O(1)$ time by simply reading `books[top]`, whereas a queue or a plain list would require walking to the end. As with the queue, a fixed array is an acceptable and simpler alternative to a linked-list stack because return history for a small library is naturally bounded in size.

4.5 Why These Structures Together

The catalog needs arbitrary search, deletion, and in-place sorting on a collection of unpredictable size, which is exactly what a linked list combined with merge sort provides. The borrow queue and return history, by contrast, only need fast end-of-structure access (append at the rear / read or remove from the front for the queue; push and read or pop from the top for the stack) on a collection with a sensible upper bound, which a plain array serves just as well as a linked list would, with less pointer bookkeeping. Using the structure that best matches each access pattern — rather than one generic container for everything — keeps every operation close to its theoretical best-case complexity.

5. Implementation

The system is implemented in a single C++ translation unit for simplicity of compilation and submission, organized into three classes — BookList, BorrowQueue, and ReturnStack — plus a console-driven main() function. BookList is built on the shared Book/Node model described below; BorrowQueue and ReturnStack are built on fixed-capacity arrays. The full source listing is provided in the Appendix; this section walks through the key design and code for each component.

5.1 Core Data Model

Three plain structs hold the project's data. Book stores a book's domain fields. Node wraps a Book together with a next pointer and is used only by the catalog's linked list. BorrowRequest is a lightweight record (book ID and borrower name) used by the array-based borrow queue.

```
struct Book
{
    int id;
    string title;
    string author;
    int year;
    bool available;
};

struct Node
{
    Book data;
    Node* next;

    Node(Book b)
    {
        data = b;
        next = nullptr;
    }
};

struct BorrowRequest
{
    int bookId;
    string borrowerName;
};
```

5.2 BookList – the Catalog

BookList encapsulates a private head pointer and exposes a public interface for every catalog operation. addBook() allocates a new node and, if the list is non-empty, walks to the current tail before linking the new node on; this keeps insertion order stable, which matters later for testing and for a predictable initial ordering before sorting.

```
void addBook(Book book)
{
    Node* newNode = new Node(book);

    if (head == nullptr)
    {
        head = newNode;
        return;
    }
```



```

Node* temp = head;
while (temp->next != nullptr)
    temp = temp->next;

temp->next = newNode;
}

```

searchBook() performs a straightforward $O(n)$ linear traversal, returning a pointer to the matching node or nullptr. This single function is reused by add (duplicate-ID checking), update, delete, borrow, and return, so the searching logic exists in exactly one place.

```

Node* searchBook(int id)
{
    Node* temp = head;
    while (temp != nullptr)
    {
        if (temp->data.id == id)
            return temp;
        temp = temp->next;
    }
    return nullptr;
}

```

deleteBook() handles the classic linked-list edge case of removing the head node separately from removing an interior or tail node, since removing the head requires updating the list's external head pointer rather than a predecessor's next pointer.

```

void deleteBook(int id)
{
    if (head == nullptr) { cout << "\nLibrary is empty.\n"; return; }

    if (head->data.id == id)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
        cout << "\nBook deleted successfully.\n";
        return;
    }

    Node* temp = head;
    while (temp->next != nullptr)
    {
        if (temp->next->data.id == id)
        {
            Node* remove = temp->next;
            temp->next = remove->next;
            delete remove;
            cout << "\nBook deleted successfully.\n";
            return;
        }
        temp = temp->next;
    }
    cout << "\nBook not found.\n";
}

```

5.3 Merge Sort on the Linked List

The three private helper functions `split()`, `merge()`, and `mergeSort()` implement the divide-and-conquer sort described in Section 4.2. `split()` uses two pointers, `slow` and `fast`, where `fast` advances two nodes for every one step of `slow`, so that when `fast` reaches the end, `slow` is at the midpoint — dividing the list without needing to know its length in advance.

```
void split(Node* source, Node*& left, Node*& right)
{
    Node* slow = source;
    Node* fast = source->next;

    while (fast != nullptr)
    {
        fast = fast->next;
        if (fast != nullptr)
        {
            slow = slow->next;
            fast = fast->next;
        }
    }

    left = source;
    right = slow->next;
    slow->next = nullptr;
}
```

`merge()` then recombines two already-sorted halves by comparing the front of each and recursively linking the smaller one's next pointer to the merge of the remainder — no temporary array is used, only pointer rewiring:

```
Node* merge(Node* a, Node* b)
{
    if (a == nullptr) return b;
    if (b == nullptr) return a;

    Node* result;
    if (a->data.year <= b->data.year)
    {
        result = a;
        result->next = merge(a->next, b);
    }
    else
    {
        result = b;
        result->next = merge(a, b->next);
    }
    return result;
}
```

`sortByYear()` ties the three helpers together with a single call, and is invoked from the menu's "Sort Books by Year" option:

```
void sortByYear()
{
    head = mergeSort(head);
    cout << "\nBooks sorted by publication year.\n";
}
```

5.4 BorrowQueue – Array-Based FIFO Borrow Requests

BorrowQueue stores up to 100 BorrowRequest records in a plain array, tracked with front and rear index counters rather than pointers. enqueue() appends a request at the rear index (reporting an error if the array is full); dequeue() reads the request at the front index and advances front, returning false if the queue is empty. Both operations are $O(1)$ because neither one scans the array.

```
void enqueue(BorrowRequest request)
{
    if (rear == 100)
    {
        cout << "\nQueue is full.\n";
        return;
    }
    requests[rear] = request;
    rear++;
}

bool dequeue(BorrowRequest& request)
{
    if (front == rear)
        return false;

    request = requests[front];
    front++;
    return true;
}
```

In the current menu, “Borrow Book” calls enqueue() to record the request and “Display Borrow Queue” reads every entry between front and rear; dequeue() is implemented and ready to use but is not yet wired to a menu option (see Section 6.4).

5.5 ReturnStack – Array-Based LIFO Return History

ReturnStack stores up to 100 Book records in a plain array, tracked with a single top index initialised to -1. push() increments top and writes the returned book into that slot, so the most recently returned book is always at the highest occupied index — an $O(1)$ operation that directly matches the “most recent first” requirement identified in Section 4.4.

```
void push(Book book)
{
    if (top == 99)
    {
        cout << "\nReturn stack is full.\n";
        return;
    }
    top++;
    books[top] = book;
}
```

displayHistory() then walks the array from top down to 0, so the newest return is always printed first, matching LIFO order without any extra sorting.

5.6 Persistence

loadFromFile() and saveToFile() give the catalog durability across runs. Each book is stored as five consecutive lines (id, title, author, year, available); loadFromFile() is called once at program start and

rebuilds the linked list from books.txt, while saveToFile() is called after every mutating menu operation so the file on disk never drifts out of sync with the in-memory list.

```
void saveToFile()
{
    ofstream file("books.txt");
    Node* temp = head;

    while (temp != nullptr)
    {
        file << temp->data.id << endl;
        file << temp->data.title << endl;
        file << temp->data.author << endl;
        file << temp->data.year << endl;
        file << temp->data.available << endl;
        temp = temp->next;
    }
    file.close();
}
```

5.7 Console Menu and main()

main() owns one instance of each class (BookList, BorrowQueue, ReturnStack) and drives a do-while loop around a numbered text menu (Add, Display, Search, Update, Delete, Borrow, Display Borrow Queue, Return, Display Return History, Sort by Year, Exit). Each branch of the loop gathers input, delegates to the appropriate class method, and calls saveToFile() whenever the catalog changes — keeping all business logic inside the classes and all I/O orchestration inside main().

This separation of concerns — data structures owning their invariants, main() owning the user interaction loop — keeps each class independently testable and made it straightforward for team members to work on different classes in parallel.

6. Analysis and Evaluation

6.1 Time and Space Complexity

The table below summarizes the complexity of every core operation implemented in the project, where n is the number of books in the catalog.

Operation	Time	Space	Notes
<code>addBook()</code>	$O(n)$	$O(1)$	Walks to tail; could be $O(1)$ if a tail pointer were cached.
<code>searchBook()</code>	$O(n)$	$O(1)$	Linear scan; reused by update, delete, borrow, return.
<code>updateBook()</code>	$O(n)$	$O(1)$	Dominated by the <code>searchBook()</code> call.
<code>deleteBook()</code>	$O(n)$	$O(1)$	Worst case: the target node is the tail.
<code>sortByYear()</code> (merge sort)	$O(n \log n)$	$O(\log n)$	Recursion depth is $\log n$; sorts in place via re-linking.
<code>enqueue()</code> / <code>dequeue()</code>	$O(1)$	$O(1)$	Index-based; bounded by the 100-slot array capacity.
<code>push()</code> (return stack)	$O(1)$	$O(1)$	Single index increment and array write.
<code>displayBooks()</code> / <code>displayQueue()</code> / <code>displayHistory()</code>	$O(n)$	$O(1)$	Must visit every stored record once to print it.

6.2 Comparison with Alternative Solutions

An array-based (vector) catalog was considered as an alternative to the linked list. A vector would give $O(1)$ random access and slightly simpler code, but insertion/deletion in the middle of a vector is $O(n)$ due to shifting elements, and the catalog would need to either reserve capacity in advance or pay amortized resize costs. The chosen singly linked list avoids any capacity assumptions and makes deletion $O(1)$ once the target node is located, at the cost of $O(n)$ search instead of $O(1)$ indexed access — an acceptable trade-off since the catalog is always searched by ID rather than by position.

For sorting, bubble sort and insertion sort were considered because they are simple to adapt to a linked list (they only need sequential access), but both run in $O(n^2)$ time in the worst case. Quicksort was also considered, but its efficient in-place partitioning relies on random access and would require either extra memory to hold indices or would lose its performance advantage on a linked list. Merge sort was selected because it needs only sequential access, guarantees $O(n \log n)$ in every case (not just on average), and can be implemented by re-linking nodes rather than copying data, which fits a singly linked list better than any of the alternatives considered.

For the borrow queue and return history, a linked-list-based queue/stack was considered as the alternative to the array-based design actually used. A linked-list version would remove the 100-entry capacity limit, but would add a pointer and a heap allocation per request. Since a small library only has a

handful of outstanding requests at once, the array-based BorrowQueue and ReturnStack were chosen for their simplicity and predictable memory use, at the cost of a hard upper bound on how many entries they can hold.

6.3 Strengths

- Guaranteed $O(n \log n)$ sort performance regardless of the initial ordering of the catalog.
- $O(1)$ enqueue and push operations for borrowing and returning, independent of catalog size.
- The catalog has no fixed capacity — it can grow until memory is exhausted, unlike the queue and stack.
- Clear separation of concerns between the three classes and the menu-driven `main()`, which made independent development and testing possible for different team members.
- Persistence ensures no data is lost when the application is closed and reopened.

6.4 Limitations

- `addBook()` is $O(n)$ because the list does not cache a tail pointer; for a very large catalog with frequent additions, caching tail would bring this down to $O(1)$.
- BorrowQueue and ReturnStack are capped at 100 entries each by their fixed array size; a library needing to track more outstanding requests or a longer history would need a larger array or a linked-list-based redesign.
- `dequeue()` and `pop()` are implemented on BorrowQueue and ReturnStack respectively, but the current menu never calls them — borrow requests and return history can only grow during a session. Wiring a “Process Borrow Request” and a “Remove Latest Return” menu option to these methods is a natural next step.
- BorrowQueue's front index only ever increases and is never reset, so repeated dequeuing (once enabled) would eventually leave unused space before front even though the logical queue is short; a circular buffer would reclaim that space.
- Persistence uses a flat, five-line-per-book text file rather than a database, so it has no transactional safety against a crash mid-write and assumes a title or author never itself contains a bare newline.
- The system is single-user; concurrent access from multiple terminals could corrupt `books.txt`.

6.5 Testing and Demonstration

The implementation was tested manually against each functional requirement from Section 2.1 using a representative set of books entered through the menu:

- Add: verified that a new book is appended and appears in Display Books; verified that adding a duplicate ID is rejected with an error message and does not modify the catalog.
- Search / Update / Delete: verified correct output for an existing ID, correct “not found” messaging for a non-existent ID, and correct field changes after Update Book.
- Borrow: verified that borrowing marks a book unavailable, that a duplicate borrow attempt on the same book is rejected, and that the request appears correctly in Display Borrow Queue in the order it was placed (confirming FIFO ordering).

- Return / History: verified that returning a book marks it available again and pushes it onto the return stack, and that Display Return History always lists the most recently returned book first (confirming LIFO ordering).
- Sort: verified that Sort Books by Year produces an ascending-by-year listing regardless of the order books were originally added, including with an odd number of books to exercise the split() midpoint logic.
- Persistence: verified that exiting and relaunching the program reloads the exact catalog state — including availability flags — from books.txt.
- Capacity limits: verified that enqueue() and push() report a clear error once their respective 100-entry arrays are full, rather than writing out of bounds.

All test cases produced the expected results, confirming that the implementation satisfies the functional requirements defined in Section 2, subject to the limitations noted in Section 6.4.

7. Appendix

7.1 Readme – How to Compile and Run

The project is a single C++ file with no external dependencies beyond the standard library.

```
g++ -std=c++17 -O2 -o library main.cpp
./library

# On first run, if books.txt does not exist, the program starts
# with an empty catalog. Books added during the session are
# saved automatically to books.txt after every change, and are
# reloaded automatically the next time the program is started.
```

7.2 books.txt File Format

Each book occupies five consecutive lines in books.txt: id, title, author, year, then available as 1 or 0. For example, two books are stored as:

```
1
Clean Code
Robert C. Martin
2008
1
2
The Pragmatic Programmer
Andrew Hunt
1999
1
```

7.3 Full Source Code

The complete source code (main.cpp) is submitted alongside this report as a separate file, and its structure is summarized here for reference:

- Book / Node / BorrowRequest – shared data model structs.
- BookList – singly linked catalog, including split()/merge()/mergeSort() and file persistence.
- BorrowQueue – array-based FIFO queue of borrow requests (capacity 100).
- ReturnStack – array-based LIFO stack of return history (capacity 100).
- main() – console menu tying the three classes together.

7.4 Project Repository

The full source code for this project is also hosted on GitHub at:

<https://github.com/abrhamnega>

7.5 Reference

rydze. Library-Management-System: Project for Data Structures and Algorithms. GitHub repository, used as a reference for general project structure.